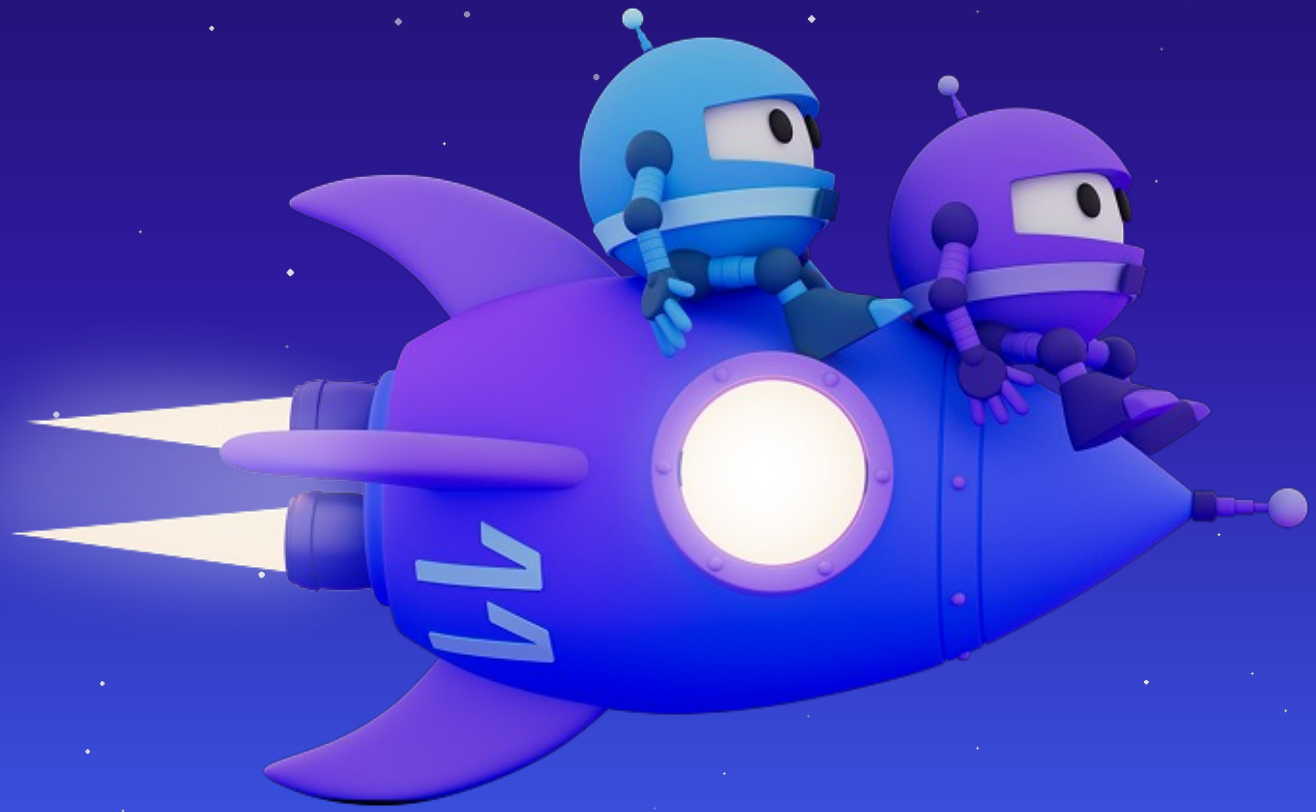
Less Chat, More Action

AI-Powered .NET MAUI Apps



Gerald Versluis

Software Engineer, Microsoft

Your users don’t want another chatbot. They want the app to act.

The phone already has the context AI needs.



Camera



Microphone



Location



Calendar



Health

What you see, what you say, where you are and what's next

Find it, do it, take me there

The user brings the intent; the app does the work



Find

"Blazor talks on Thursday."

Sessions opens, already filtered

find_sessions



Act

"Save all the Aspire sessions."

Every heart fills in, My Schedule opens

save_sessions



Take me there

"Who's giving the Blazor talk?"

The speaker page opens

open_speaker

Live demo

Plan my conference



01 The building blocks



One API, any model

One IChatClient for cloud, local and on-device models.

Your view models depend on the interface, never on a provider.

Tool calling and telemetry plug in as middleware.

```
using Microsoft.Extensions.AI;

// Your code only ever sees IChatClient
public class SessionAssistant(IChatClient chat)
{
    public async Task<string> AskAsync(string q)
    {
        var response = await chat.GetResponseAsync(q);
        return response.Text;
    }
}

// Add capabilities as middleware
var client = innerClient.AsBuilder()
    .UseFunctionInvocation()
    .UseOpenTelemetry()
    .Build();
```

Three packages, one stack

Each layer can be swapped on its own

Your app — Pages, view models and services you already have

AI attributes — [ExportAIFunction] exposes app actions as tools

Microsoft.Extensions.AI — IChatClient, tool calling, telemetry

Essentials.AI
On-device: Apple, Gemini Nano, Windows AI

Cloud providers
Microsoft Foundry, OpenAI and more

02 Essentials.AI



Essentials, but for AI

What we did for sensors, now for models

2018

Xamarin.Essentials

One cross-platform API for device features: geolocation, battery, preferences, secure storage and more.

2022

Part of .NET MAUI

Essentials moved into .NET MAUI itself. No extra package, the same APIs on every platform.

Now

Essentials.AI

One API for the model already on the device: Apple Intelligence, Gemini Nano and Windows AI, behind IChatClient.

Get started

One package and one registration in MauiProgram.

The rest of your app only sees IChatClient.

Always check the model is ready. Not every device has one.

```
// 1. Add the package, opt in to preview
// dotnet add package
//     Microsoft.Maui.Essentials.AI --prerelease
// <NoWarn>$(NoWarn);MAUIAI0001</NoWarn>

// 2. MauiProgram.cs: one #if, in one place
#if IOS || MACCATALYST
builder.Services.AddSingleton<IChatClient>(
    new AppleIntelligenceChatClient());
#elif ANDROID
builder.Services.AddSingleton<IChatClient>(
    new GeminiNanoChatClient());
#endif

// 3. Before offering AI: is the model ready?
var status = await AppleIntelligenceChatClient
    .GetModelStatusAsync(); // or GeminiNano ...
ShowAsk = status.Availability is
    ChatModelAvailability.Available;
```

Bring your own model

Three ways to get a model, one IChatClient

Built into the OS

Apple Intelligence, Gemini Nano, Windows AI

Via Essentials.AI. Nothing to ship with your app, and nothing leaves the device.

```
new AppleIntelligenceChatClient()
```

Bundled model

Any ONNX model, like Phi or Llama

Via ONNX Runtime GenAI. Still on the device, but you ship or download the model.

```
new OnnxRuntimeGenAIChatClient(…)
```

In the cloud

Microsoft Foundry, OpenAI and more

The largest models. Needs a network and a backend you control for the key.

```
chatClient.AsIChatClient()
```

← More private, free per call

More capable →

All three implement IChatClient: swapping is one registration line

On-device or cloud: same code

Pick the trade-off per feature, not per app

On-device

Essentials.AI

Private, offline, no API key and no per-token cost. Smaller model.

Cloud

Microsoft Foundry, OpenAI

Larger models for harder tasks. Needs a network, a key and a budget.

On-device AI on every platform

One IChatClient, three native models

Platform	Chat	Tool calling
iOS, macOS, Mac Catalyst	Apple Intelligence	Yes
Android	Gemini Nano	Nano V4 models
Windows	Windows AI	Not yet

Needs a supported device with the model enabled. Windows: Copilot+ PC and an MSIX-packaged app. Embeddings: Apple only. Apple image input and token usage: OS 27.

03 AI attributes



Methods as tools

[ExportAIFunction] turns a method into a tool.

[Description] is what the model reads: when to use it, and what not to do.

A source generator writes the calling code at compile time: no reflection when the tool runs.

```
// AskTools.cs, from the app
[ExportAIFunction("open_speaker")]
[Description("Open a speaker profile. Use " +
    "for 'who is giving' a talk, or opening " +
    "a named person. Never invent a name. " +
    "Call once.")]
public string OpenSpeaker(
    [Description("Talk title/topic if asking " +
        "who gives it; empty for a direct " +
        "person name")] string query = "",
    [Description("Weekday or date copied " +
        "from the user, or empty")]
    string day = "",
    [Description("Person name copied from " +
        "the user, or empty if unknown")]
    string speaker = "") ⇒
    RequestAction("open_speaker",
        query, day, speaker);
```

Collect the tools, wire them up

```
// Collect every tool in AskTools into one context
[AIToolSource(typeof(AskTools))]
public partial class AskToolContext : AIToolContext;

// Offer only the tools this request needs (excerpt)
var functions = AskToolContext.Default.Tools.Cast<AIFunction>()
    .Where(f => f.Name == "lookup_sessions" ||
        AskRequest.CanOfferAction(request, f.Name switch {
            "find_sessions" => AskAction.FindSessions,
            "request_removal" => AskAction.RemoveSessions,
            /*  ...3 more mappings */ }));

using var client = chatClient.AsBuilder()
    .UseFunctionInvocation(configure: invocation => {
        invocation.AllowConcurrentInvocation = false;
        invocation.MaximumIterationsPerRequest = 8; })
    .Build(scope.ServiceProvider);
```

The app acts

Every tool stages an action; it doesn't touch the UI itself.

The app then runs it through the same view models and services the screens use.

So the heart fills in and the page opens, exactly as if you tapped.

```
// AskTools.cs (excerpt)
public string RequestAction(string action,
    string query = "", string day = "",
    string speaker = "")
{
    var kind = action switch { /* ... */ };
    // Search terms must appear in the request
    var intent = AskRequest.Ground(GetDocument(),
        request, new(kind, query, day, speaker));
    AskRequest.ValidateIntent(GetDocument(),
        request, intent);
    // Resolve against the agenda the screens show
    var next = AskRequest.Resolve(GetDocument(),
        intent, zone, savedIds);
    // ... conflicting-action checks omitted
    Pending = next;
    return "Action staged ... The application " +
        "handles choices and confirmation.";
}
```

Live demo

Ask, and it acts



04 Users in control



Propose, then confirm

ApprovalRequired = true: the model can ask, never execute.

The app shows exactly what will be removed.

Confirming runs plain C#, with no further model turn.

```
[ExportAIFunction("request_removal",
    ApprovalRequired = true)]
[Description("Request removal of saved sessions. " +
    "The app must display an exact preview for " +
    "user approval. Never confirm on behalf of " +
    "the user.")]
public string RequestRemoval(string query = "",
    string day = "", string speaker = "") ⇒ ...

// OnDeviceAsk.cs: the call comes back for approval
var approvals = response.Messages
    .SelectMany(m ⇒ m.Contents)
    .OfType<ToolApprovalRequestContent>().ToArray();
if (approvals.Length ≠ 1 || approvals[0].ToolCall
        is not FunctionCallContent call) throw ...;
tools.ReadRemovalPreview(call); // preview only
```

Where to draw the line

Four rules for an assistant that acts



Validate every request

Treat tool arguments like user input: check IDs, ranges and permissions



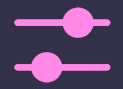
Confirm destructive actions

ApprovalRequired on delete, pay, send and anything hard to undo



Say what you can’t do

Return a clear message from the tool so the model doesn't guess



Expose only what’s needed

Curate tools per feature with a tool context

Semantic search

A built-in model turns text into vectors, on the device.

Similar meaning, nearby vectors, even without shared words.

Index once; each search compares vectors in memory.

```
// Example, not from the demo app (Apple only)
// Built-in English sentence embeddings
using var embedder = new NLEmbeddingGenerator();

// Index the session descriptions once
var vectors = await embedder.GenerateAsync(
    sessions.Select(s ⇒ s.Description));

// Search by meaning, not keywords
var q = await embedder.GenerateVectorAsync(
    "talks about AI on phones");

var best = sessions.Zip(vectors).MaxBy(p ⇒
    TensorPrimitives.CosineSimilarity(
        q.Span, p.Second.Vector.Span));
```

Ideas we haven’t built yet

Same pattern, very different experiences

Find any setting

“Where do I switch to dark mode?”

AI wayfinding (draft PR #495)

Snap a slide, find it

A photo from a watch party

On-device image input (OS 27, Android)

Settle a clash

“These two overlap. Which one live?”

Overlap check as a tool

Build a learning path

“New to Aspire. Where do I start?”

Topic tools, ordered by level

Brief me first

“What should I know before this talk?”

A summary of the abstract

Read it in my language

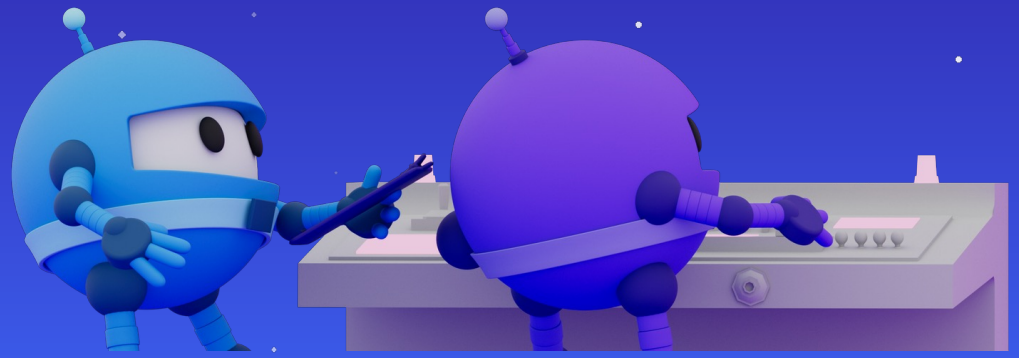
Abstracts and bios, translated

On-device, where the model supports it

What if the UI follows you?

Today the app knows what you want and opens the right screen. Next, it could build the screen you need: your view, for your intent.

First steps in maui-labs: AI wayfinding (draft PR #495)



Recap

An assistant that does more than talk

- **Microsoft.Extensions.AI:** one IChatClient for every model
- **Essentials.AI:** private, offline AI on the device
- **AI attributes:** app actions become tools with one attribute
- **Propose, then confirm: keep users in control**

Try it today

Experimental packages: APIs may change

- **Packages:** Microsoft.Maui.Essentials.AI, Microsoft.Maui.AI.Attributes
- **Labs repo:** github.com/dotnet/maui-labs
- **Demo app:** github.com/jfversluis/dotnetconf-app
- **Garden sample:** AIExtensions.Sample.Garden in the Labs repo
- **BaristaNotes:** github.com/davidortinau/BaristaNotes
- **Docs:** learn.microsoft.com/dotnet/maui/ai

Thank you

